Aror University of Art, Architecture, Design & Heritage Sukkur.

Department of Artificial Intelligence

Data Structures (Fall-2026)

LAB No. 6

Objective of Lab No. 6:
After performing lab1, students will be able to:

- Understand the structure of a singly linked list and the role of nodes and links.
- Create a singly linked list using a user-defined Node class.
- Traverse a linked list and display all nodes in sequence.
- Insert new nodes at the beginning, end, and a specified position.
- Update node data using a known position and after locating a value.
- Delete nodes from the beginning, end, and a specified position.
- Understand how pointer/link changes differ from array element shifting.
- Analyze the time and auxiliary space complexity of linked-list operations using Big-O notation.
- Develop programs that use traversal when a direct index is not available.
- Measure selected execution times using time.perf_counter() and distinguish measured runtime from theoretical complexity.

Note: This lab uses a singly linked list implemented with a user-defined Node class. Each node stores data and a reference named next to the following node. The first node is referenced by head, and the final node points to None. Unlike Python lists, linked lists do not provide direct indexed access; therefore, reaching a particular position normally requires traversal from head.

Core Operations Covered:

Operation	Typical Time	Auxiliary Space	Main Idea
Create / build list	O(n)	O(n) total nodes	Create and connect nodes
Traverse	O(n)	O(1) auxiliary	Visit nodes from head to None
Insert at beginning	O(1)	O(1) auxiliary	New node points to current head
Insert at end	O(n)	O(1) auxiliary	Traverse to last node

			without tail
Insert at position	O(n)	O(1) auxiliary	Traverse to predecessor, then relink
Update known position	O(n)	O(1) auxiliary	Traverse to required node
Update by value	O(n)	O(1) auxiliary	Search, then change data
Delete at beginning	O(1)	O(1) auxiliary	Move head to second node
Delete at end	O(n)	O(1) auxiliary	Traverse to second-last node
Delete at position	O(n)	O(1) auxiliary	Traverse to predecessor, then relink

Lab Tasks:

The following programs are divided into four sections. Each topic contains at least two practice programs, two lab-submission programs, and one scenario-based program. Students should run the programs, study the link changes, verify the output, and write the time and auxiliary space complexity.

Part A — Creating a Singly Linked List

Creation means defining a Node, creating node objects, connecting them through next references, assigning head, and traversing the resulting list. The examples progress from manually connecting nodes to building a list through a function.

Practice Program 1 — Create and Display Three Nodes

Create three nodes manually, connect them, and display the linked list.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

first = Node(10)
second = Node(20)
third = Node(30)
```

```
first.next = second
second.next = third

head = first

current = head
while current is not None:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
10 -> 20 -> 30 -> None
```

Explanation: The nodes are created separately and then connected using next. head points to the first node, so traversal begins at 10 and follows each link until None.
Time Complexity: O(n)
Auxiliary Space Complexity: O(1) auxiliary after node creation; O(n) total storage for the n nodes
Complexity Analysis: Traversal visits every node once. The list itself requires O(n) storage because n Node objects are created, but the traversal uses only a constant number of extra references.

Practice Program 2 — Create a Linked List Using a Loop

Create five nodes from a list of values and connect each new node after the current last node.
Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

values = [5, 15, 25, 35, 45]
head = None
tail = None

for value in values:
    new_node = Node(value)

    if head is None:
        head = new_node
        tail = new_node
    else:
        tail.next = new_node
        tail = new_node

current = head
while current:
```

```
        print(current.data, end=" -> ")
        current = current.next
    print("None")
```

Expected Output (example):

```
5 -> 15 -> 25 -> 35 -> 45 -> None
```

Explanation: head identifies the first node and tail identifies the last node. Because tail is maintained, each new node can be attached directly without traversing the existing list.
Time Complexity: O(n) for creating n nodes and displaying them
Auxiliary Space Complexity: O(n) total list storage; O(1) auxiliary space beyond the nodes
Complexity Analysis: Each input value creates one node and traversal processes each node once. Maintaining tail makes each append O(1), so building the list is O(n).

Lab Submission Program 1 — Create a Linked List from User Input

Ask the user for the number of nodes, accept their values, create the linked list, and display it.
Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = None
tail = None

n = int(input("Enter number of nodes: "))

for i in range(n):
    value = int(input(f"Enter value {i + 1}: "))
    new_node = Node(value)

    if head is None:
        head = new_node
        tail = new_node
    else:
        tail.next = new_node
        tail = new_node

print("Linked List:")
current = head
while current is not None:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
Enter number of nodes: 4
Enter value 1: 11
Enter value 2: 22
Enter value 3: 33
Enter value 4: 44
Linked List:
11 -> 22 -> 33 -> 44 -> None
```

Explanation: The program creates a node for every input value. tail is used to connect the next node efficiently, while head remains the entry point to the complete list.
Time Complexity: O(n)
Auxiliary Space Complexity: O(n) total storage; O(1) auxiliary space beyond created nodes
Complexity Analysis: Creating n nodes requires O(n) time and O(n) storage. Because tail is maintained, no repeated traversal is needed during construction.

Lab Submission Program 2 — Create List and Count Nodes

Create a linked list and determine its length by traversing from head to None.
Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = None
tail = None

values = [12, 24, 36, 48, 60]

for value in values:
    new_node = Node(value)

    if head is None:
        head = new_node
        tail = new_node
    else:
        tail.next = new_node
        tail = new_node

count = 0
current = head

while current is not None:
    count += 1
    current = current.next

print("Linked List:")
current = head
while current:
```

```
        print(current.data, end=" -> ")
        current = current.next
    print("None")
    print("Number of Nodes:", count)
```

Expected Output (example):

```
Linked List:
12 -> 24 -> 36 -> 48 -> 60 -> None
Number of Nodes: 5
```

Explanation: Two traversals are used: the first displays the list and the second counts the nodes. A linked list has no built-in direct index access in this implementation, so counting requires visiting each node.
Time Complexity: O(n)
Auxiliary Space Complexity: O(1) auxiliary space
Complexity Analysis: The list contains n nodes, and the traversal examines each node once for counting. The number of extra variables remains constant.

Scenario-Based Program — Employee Record Chain

A small company stores employee IDs in the order in which employees joined. Implement a singly linked list for the IDs, display the chain, and report the number of employees.
Program:

```
class Node:
    def __init__(self, employee_id):
        self.data = employee_id
        self.next = None

employee_ids = [101, 104, 108, 112, 115, 119]
head = None
tail = None

for employee_id in employee_ids:
    new_node = Node(employee_id)

    if head is None:
        head = new_node
        tail = new_node
    else:
        tail.next = new_node
        tail = new_node

count = 0
current = head

print("Employee ID Chain:")
while current:
    print(current.data, end=" -> ")
```

```
        count += 1
        current = current.next

    print("None")
    print("Total Employees:", count)
```

Expected Output (example):

```
Employee ID Chain:
101 -> 104 -> 108 -> 112 -> 115 -> 119 -> None
Total Employees: 6
```

Explanation: Each employee ID is represented by one node. The links preserve joining order. The program demonstrates creation, traversal, and counting in one practical application.
Time Complexity: O(n)
Auxiliary Space Complexity: O(n) total storage; O(1) auxiliary space
Complexity Analysis: Every employee ID is processed once during creation and once during traversal. Thus the running time grows linearly with the number of employees.

Part B — Insertion in a Singly Linked List

Insertion adds a new node without shifting existing data. The important operation is changing next references. Insertion at the beginning is O(1) because no traversal is required. Insertion at the end or at an arbitrary position is O(n) in this implementation because the required location may have to be reached by traversal.

Practice Program 1 — Insert at the Beginning

Insert a new node before the current head.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

first = Node(20)
second = Node(30)
third = Node(40)
first.next = second
second.next = third

head = first

new_node = Node(10)
new_node.next = head
head = new_node

current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
10 -> 20 -> 30 -> 40 -> None
```

Explanation: The new node points to the old head, and then head is updated to the new node. No existing node needs to move.
Time Complexity: O(1)
Auxiliary Space Complexity: O(1) auxiliary
Complexity Analysis: Only two link/reference assignments are required, independent of the number of nodes.

Practice Program 2 — Insert at the End

Insert a new node after the current last node without maintaining a tail reference.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)

new_node = Node(40)

current = head
while current.next is not None:
    current = current.next

current.next = new_node

current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
10 -> 20 -> 30 -> 40 -> None
```

Explanation: The program traverses to the last node, where next is None, and then changes that next reference to the new node.
Time Complexity: O(n)
Auxiliary Space Complexity: O(1) auxiliary
Complexity Analysis: In the worst case, traversal visits all n nodes before the insertion point is reached. The new node itself is part of the list storage, not auxiliary space.

Lab Submission Program 1 — Insert at a Specified Position

Insert a new value at a zero-based position. Position 0 means insertion at the beginning.
Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)
head.next.next.next = Node(40)
```

```
value = 25
position = 2

new_node = Node(value)

if position == 0:
    new_node.next = head
    head = new_node
else:
    current = head

    for _ in range(position - 1):
        if current is None:
            break
        current = current.next

    if current is None:
        print("Invalid position")
    else:
        new_node.next = current.next
        current.next = new_node

print("Updated Linked List:")
current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
Updated Linked List:
10 -> 20 -> 25 -> 30 -> 40 -> None
```

Explanation: To insert at position 2, the program first reaches the node at position 1 (20). The new node is linked between 20 and 30 by changing two next references.
Time Complexity: O(n) worst case
Auxiliary Space Complexity: O(1) auxiliary
Complexity Analysis: The predecessor may be near the end of the list, so traversal can take O(n). Once the predecessor is found, insertion itself is O(1).

Lab Submission Program 2 — Insert Multiple Nodes

Insert one node at the beginning and another at the end, then display the final linked list.
Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
head = Node(20)
head.next = Node(30)
head.next.next = Node(40)

# Insert 10 at beginning
new_start = Node(10)
new_start.next = head
head = new_start

# Insert 50 at end
new_end = Node(50)
current = head
while current.next is not None:
    current = current.next
current.next = new_end

print("After Insertions:")
current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
After Insertions:
10 -> 20 -> 30 -> 40 -> 50 -> None
```

Explanation: The first insertion changes head directly. The second insertion requires traversal to the last node because no tail reference is maintained.
Time Complexity: O(n)
Auxiliary Space Complexity: O(1) auxiliary
Complexity Analysis: The beginning insertion is O(1), while the end insertion is O(n). The combined sequence is therefore O(n).

Scenario-Based Program — Hospital Patient Queue

A hospital maintains patient token numbers in a linked list. A new emergency patient with token 50 must be inserted at the beginning, and a regular patient with token 85 must be added at the end.

Program:

```
class Node:
    def __init__(self, token):
        self.data = token
        self.next = None

head = Node(60)
head.next = Node(70)
```

```
head.next.next = Node(80)

# Emergency patient: insert at beginning
emergency = Node(50)
emergency.next = head
head = emergency

# Regular patient: insert at end
regular = Node(85)
current = head
while current.next is not None:
    current = current.next
current.next = regular

print("Patient Token List:")
current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
Patient Token List:
50 -> 60 -> 70 -> 80 -> 85 -> None
```

Explanation: The emergency token is inserted at the front without traversal. The regular token is inserted at the end after traversing to the last node. This illustrates why insertion cost depends on the location.
Time Complexity: O(n) overall
Auxiliary Space Complexity: O(1) auxiliary
Complexity Analysis: Beginning insertion is O(1), but end insertion is O(n) without a tail reference. The complete scenario is dominated by the O(n) operation.

Part C — Updation in a Singly Linked List

Updation changes the data stored in an existing node. Unlike an array, a linked list does not provide direct indexed access. Even if a position is known, the program generally traverses from head to reach that node. If the target value is known but its position is unknown, a search is also required.

Practice Program 1 — Update Node at a Position

Update the data at position 2 by traversing from head.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)
head.next.next.next = Node(40)

position = 2
new_value = 35

current = head
for _ in range(position):
    if current is None:
        break
    current = current.next

if current is not None:
    current.data = new_value

print("Updated Linked List:")
current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
Updated Linked List:
10 -> 20 -> 35 -> 40 -> None
```

Explanation: The node at position 2 is reached by following two next references. Its data is then replaced with 35. No links are changed.

Time Complexity: O(n) worst case
Auxiliary Space Complexity: O(1) auxiliary
Complexity Analysis: Reaching a position near the end may require visiting almost all nodes. Once the node is reached, changing data is O(1).

Practice Program 2 — Update a Node After Searching for a Value

Search for a student name and replace it with a corrected name.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node("Ali")
head.next = Node("Sara")
head.next.next = Node("Ahmed")
head.next.next.next = Node("Hina")

old_name = "Ahmed"
new_name = "Ahmad"

current = head
while current is not None:
    if current.data == old_name:
        current.data = new_name
        break
    current = current.next

print("Updated Names:")
current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
Updated Names:
Ali -> Sara -> Ahmad -> Hina -> None
```

Explanation: The program does not know Ahmed's position in advance. It searches node by node until the value is found, then changes only that node's data.
Time Complexity: O(n) worst case
Auxiliary Space Complexity: O(1) auxiliary
Complexity Analysis: Searching may inspect all n nodes in the worst case. The update after finding the node takes constant time.

Lab Submission Program 1 — Update Multiple Positions

Update two nodes at specified zero-based positions in the same linked list.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)
head.next.next.next = Node(40)
head.next.next.next.next = Node(50)

updates = [(1, 25), (3, 45)]

for position, new_value in updates:
    current = head

    for _ in range(position):
        if current is None:
            break
        current = current.next

    if current is not None:
        current.data = new_value

print("Updated Linked List:")
current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
Updated Linked List:
10 -> 25 -> 30 -> 45 -> 50 -> None
```

Explanation: Each update starts from head because the linked list does not support direct indexing. The program reaches each required node and changes only its data field.

Time Complexity: O(n) for a fixed number of updates

Auxiliary Space Complexity: O(1) auxiliary

Complexity Analysis: Each update can require O(n) traversal in the worst case. Because only a fixed number of updates are performed, the overall complexity remains O(n).

Lab Submission Program 2 — Search and Update Product Price

Find a product by name and update its price without changing the links.

Program:

```
class Node:
    def __init__(self, name, price):
        self.name = name
        self.price = price
        self.next = None

first = Node("Keyboard", 2500)
second = Node("Mouse", 1200)
third = Node("Monitor", 28000)

first.next = second
second.next = third
head = first

target = "Mouse"
new_price = 1500

current = head
while current is not None:
    if current.name == target:
        current.price = new_price
        break
    current = current.next

print("Product List:")
current = head
while current:
    print(current.name, ":", current.price)
    current = current.next
```

Expected Output (example):

```
Product List:
Keyboard : 2500
Mouse : 1500
Monitor : 28000
```

Explanation: The program searches for the product name. Once Mouse is found, only its price is changed. The next references remain unchanged.
Time Complexity: O(n) worst case
Auxiliary Space Complexity: O(1) auxiliary
Complexity Analysis: Finding the product may require traversing the entire list. The actual price update is O(1).

Scenario-Based Program — Correcting Student Attendance Record

A classroom linked list contains student names. The teacher discovers that 'Maham' was entered incorrectly as 'Meham'. Locate the record and correct it.

Program:

```
class Node:
    def __init__(self, name):
        self.data = name
        self.next = None

head = Node("Ayesha")
head.next = Node("Maham")
head.next.next = Node("Sana")
head.next.next.next = Node("Hira")

old_name = "Maham"
correct_name = "Meham"

current = head
while current is not None:
    if current.data == old_name:
        current.data = correct_name
        break
    current = current.next

print("Corrected Attendance List:")
current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
Corrected Attendance List:
Ayesha -> Meham -> Sana -> Hira -> None
```

Explanation: The scenario represents a realistic record correction. Because the student's position is not supplied, the program searches by name and then updates the matching node.
Time Complexity: O(n) worst case
Auxiliary Space Complexity: O(1) auxiliary
Complexity Analysis: The search can require a complete traversal. Only one node's data is changed, so no additional list structure is required.

Part D — Deletion in a Singly Linked List

Deletion removes a node from the logical chain by changing links. The node itself does not require shifting of other nodes. Deletion at the beginning is O(1). Deletion at the end or a specified position is O(n) in this implementation because the program must usually find the relevant predecessor node.

Practice Program 1 — Delete from the Beginning

Remove the first node by moving head to the second node.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)

removed = head.data
head = head.next

print("Removed:", removed)
print("Updated Linked List:")
current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
Removed: 10
Updated Linked List:
20 -> 30 -> None
```

Explanation: head originally points to 10. Assigning head = head.next makes 20 the first node. No other links need to be changed.

Time Complexity: O(1)

Auxiliary Space Complexity: O(1) auxiliary

Complexity Analysis: Only one reference assignment is needed, independent of list size.

Practice Program 2 — Delete from the End

Delete the final node by finding the second-last node.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)
head.next.next.next = Node(40)

current = head
while current.next.next is not None:
    current = current.next

removed = current.next.data
current.next = None

print("Removed:", removed)
print("Updated Linked List:")
current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
Removed: 40
Updated Linked List:
10 -> 20 -> 30 -> None
```

Explanation: The program stops at the second-last node. Its next reference is changed from the last node to None.
Time Complexity: O(n)
Auxiliary Space Complexity: O(1) auxiliary
Complexity Analysis: Without a tail and previous-node reference, reaching the second-last node requires traversal. Thus deletion at the end is O(n).

Lab Submission Program 1 — Delete at a Specified Position

Delete the node at a zero-based position. Handle position 0 separately.
Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
```

```
head.next.next = Node(30)
head.next.next.next = Node(40)
head.next.next.next.next = Node(50)

position = 2

if head is None:
    print("List is empty")
elif position == 0:
    removed = head.data
    head = head.next
    print("Removed:", removed)
else:
    current = head

    for _ in range(position - 1):
        if current is None:
            break
        current = current.next

    if current is None or current.next is None:
        print("Invalid position")
    else:
        removed = current.next.data
        current.next = current.next.next
        print("Removed:", removed)

print("Updated Linked List:")
current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
Removed: 30
Updated Linked List:
10 -> 20 -> 40 -> 50 -> None
```

Explanation: To delete position 2, the program reaches position 1 (20), saves the value of the next node, and skips that node by setting current.next to current.next.next.
Time Complexity: O(n) worst case
Auxiliary Space Complexity: O(1) auxiliary
Complexity Analysis: Finding the predecessor may require traversal through O(n) nodes. The actual unlink operation is O(1).

Lab Submission Program 2 — Delete a Node by Value

Search for a given value and delete its first occurrence.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(15)
head.next = Node(25)
head.next.next = Node(35)
head.next.next.next = Node(45)

target = 35

if head is not None and head.data == target:
    head = head.next
else:
    current = head

    while current is not None and current.next is not None:
        if current.next.data == target:
            current.next = current.next.next
            break
        current = current.next

print("After Deletion:")
current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
After Deletion:
15 -> 25 -> 45 -> None
```

Explanation: The program searches for the node before the target. When 35 is found in current.next, the link skips over that node and points directly to 45.
Time Complexity: O(n) worst case
Auxiliary Space Complexity: O(1) auxiliary
Complexity Analysis: Finding the target can require visiting every node. The deletion itself only changes one next reference.

Scenario-Based Program — Removing a Student from a Course List

A course linked list contains student SAP IDs. Student 1030 has dropped the course. Remove that student's node and display the remaining list.
Program:

```
class Node:
    def __init__(self, sap_id):
        self.data = sap_id
        self.next = None

head = Node(1010)
head.next = Node(1020)
head.next.next = Node(1030)
head.next.next.next = Node(1040)
head.next.next.next.next = Node(1050)

target = 1030

if head is not None and head.data == target:
    head = head.next
else:
    current = head
    while current is not None and current.next is not None:
        if current.next.data == target:
            current.next = current.next.next
            break
        current = current.next

print("Remaining Student SAP IDs:")
current = head
while current:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

Expected Output (example):

```
Remaining Student SAP IDs:
1010 -> 1020 -> 1040 -> 1050 -> None
```

Explanation: The program locates SAP ID 1030 and unlinks its node. The other nodes remain intact; no element shifting is required.
Time Complexity: O(n) worst case
Auxiliary Space Complexity: O(1) auxiliary
Complexity Analysis: The target may be near the end or absent, requiring traversal of the list. Therefore the search-and-delete operation is O(n) in the worst case.

Algorithms for Core Linked-List Operations

Students should understand the link changes before writing code. The following algorithms summarize the four main operations.

Algorithm 1 — Create a Singly Linked List

```
CREATE_LIST(values)
    head ← None
    tail ← None

    FOR each value in values
        newNode ← create Node(value)

        IF head = None
            head ← newNode
            tail ← newNode
        ELSE
            tail.next ← newNode
            tail ← newNode

    RETURN head
```

Algorithm 2 — Insert at Beginning

```
INSERT_BEGINNING(head, value)
    newNode ← create Node(value)
    newNode.next ← head
    head ← newNode
    RETURN head
```

Algorithm 3 — Insert at Position

```
INSERT_POSITION(head, value, position)
    newNode ← create Node(value)

    IF position = 0
        newNode.next ← head
        head ← newNode
        RETURN head

    current ← head
    REPEAT position - 1 times
        current ← current.next

    newNode.next ← current.next
    current.next ← newNode
    RETURN head
```

Algorithm 4 — Update by Value

```
UPDATE_VALUE(head, oldValue, newValue)
    current ← head

    WHILE current ≠ None
        IF current.data = oldValue
            current.data ← newValue
            RETURN head
        current ← current.next

    RETURN head
```

Algorithm 5 — Delete by Value

```
DELETE_VALUE(head, target)
    IF head = None
        RETURN head

    IF head.data = target
        head ← head.next
        RETURN head

    current ← head

    WHILE current.next ≠ None
        IF current.next.data = target
            current.next ← current.next.next
            RETURN head
        current ← current.next

    RETURN head
```

Dry Run Example — Insertion at Position 2

Initial list:

```
head
 |
 v
10 -> 20 -> 30 -> 40 -> None
```

Insert 25 at position 2:

```
Step 1: Reach node 20 (the predecessor of position 2)
Step 2: newNode = 25
Step 3: newNode.next = 30
Step 4: 20.next = newNode
```

```
Final:
10 -> 20 -> 25 -> 30 -> 40 -> None
```

Dry Run Example — Deletion by Value

Delete 30 from:

```
10 -> 20 -> 30 -> 40 -> None
```

- Start at 10; 10 is not 30.
- Check 20.next; its data is 30.
- Set 20.next = 30.next.
- The node containing 30 is no longer part of the chain.
- Final list: 10 -> 20 -> 40 -> None.

Linked List vs Array/List Operations

Operation	Array/List	Singly Linked List	Key Difference	Typical Auxiliary Space
Access known position	O(1)	O(n)	Array has direct indexing; linked list needs traversal	O(1)
Insert beginning	O(n)	O(1)	Array shifts elements; linked list changes head	O(1)
Insert end	O(1) amortized	O(n) without tail	Linked list must find last node unless tail is maintained	O(1)
Insert middle	O(n)	O(n)	Both may require reaching the location; array shifts, list relinks	O(1)
Delete beginning	O(n)	O(1)	Array shifts; linked list moves head	O(1)
Delete end	O(1)	O(n) without tail	Singly list must find predecessor	O(1)
Delete middle	O(n)	O(n)	Linked list relinks after traversal; array shifts	O(1)
Update by position	O(1)	O(n)	Linked list must traverse to the position	O(1)

Complexity Reference

Operation	Typical Time	Auxiliary Space	Reason
Create/build using tail	O(n)	O(1) beyond nodes	Each input creates one node and is linked directly

Traverse	O(n)	O(1)	Every node is visited once
Insert at beginning	O(1)	O(1)	Change newNode.next and head
Insert at end without tail	O(n)	O(1)	Traverse to last node
Insert at specified position	O(n) worst case	O(1)	Traverse to predecessor
Update by position	O(n) worst case	O(1)	Traverse from head
Update by value	O(n) worst case	O(1)	Search through nodes
Delete at beginning	O(1)	O(1)	Move head to next node
Delete at end without tail	O(n)	O(1)	Find second-last node
Delete at position/value	O(n) worst case	O(1)	Find target/predecessor then relink

Important Complexity Observation:

A linked list can insert or delete a node in O(1) once the correct predecessor/location reference is already available. However, in these introductory programs the required location is usually found by traversing from head, so insertion/deletion at an arbitrary position is reported as O(n) in the worst case. This distinction is important when comparing linked lists with arrays.

Execution-Time Measurement Note:

time.perf_counter() measures observed elapsed execution time using a high-resolution performance counter. Measured values depend on hardware, operating-system activity, Python version, and background processes. Use execution time to observe behavior, but determine Big-O from the number of nodes visited and link operations performed.

```
import time

start_time = time.perf_counter()

# linked-list operation

end_time = time.perf_counter()
```

```
print("Execution Time:", end_time - start_time, "seconds")
```

Lab Exercise / Submission Tasks

Complete the following submission programs independently. For every task, run the program, include the output, write the algorithm in your notebook, and provide a short dry run. State and justify time and auxiliary space complexity.

- Task 1: Create a singly linked list containing at least 8 integer values. Display all nodes from head to None.
- Task 2: Create a linked list of at least 6 student names using a Node class and display the list.
- Task 3: Insert a new node at the beginning of a list containing at least 7 values. Draw the list before and after insertion.
- Task 4: Insert a new node at the end without using a tail reference. Explain why traversal is required.
- Task 5: Insert a value at a user-specified valid position. Test positions near the beginning, middle, and end.
- Task 6: Update a node at a specified position. Explain why linked-list update is not direct indexed access.
- Task 7: Search for a student name and update it to a corrected name. Test both found and not-found cases.
- Task 8: Delete the first node from a linked list and display the removed value and final list.
- Task 9: Delete a node from a specified position. Test deletion near the beginning, middle, and end.
- Task 10: Search for a target value and delete its first occurrence. Also test a target that does not exist.

Scenario-Based: University Course Enrollment

A university stores the SAP IDs of enrolled students in a singly linked list. New students can be added, incorrect records can be updated, and students who withdraw must be removed.
Requirements:

- Create at least 8 SAP IDs in a linked list.
- Insert one new SAP ID at the beginning and another at the end.
- Update one SAP ID after searching for an old value.
- Delete one SAP ID after searching for the target.
- Display the list before and after all operations.
- State the time complexity of each operation and explain the number of nodes traversed.
- Draw at least one before/after link diagram for an insertion and one for a deletion.

Scenario-Based: Hospital Patient Management

A hospital maintains patient token numbers in a singly linked list. Emergency patients are inserted at the beginning, regular patients are added at the end, patient information may be

corrected, and a completed patient's token is deleted.
Requirements:

- Create at least 7 patient token numbers.
- Insert an emergency token at the beginning.
- Insert a regular token at the end.
- Update one token after locating it.
- Delete one token after locating it.
- Display the final linked list.
- Provide a short complexity comparison for beginning versus end operations.

Scenario-Based: Inventory Product Chain

A store keeps product IDs in a singly linked list. A new product is added, an existing product price is corrected, and a discontinued product is removed.
Requirements:

- Represent at least 6 products using nodes containing product ID and price.
- Insert a new product at a specified position.
- Search for a product ID and update its price.
- Search for a discontinued product ID and delete that node.
- Display the complete product chain before and after the operations.
- State time and auxiliary space complexity for each operation.